

Benoit's note – the two remaining bullets

Stop the Health page writing indicator values into the configuration, and stop the “Applicable flight” popup opening. Two edits, two files. Everything else in that note is already in: the configuration is still created, and Part D makes the analysis read the deviation.

He says	Where	Status
A new configuration is created when you update an indicator	–	Background, nothing to do
It won't be possible to modify the indicator's values directly – switch off this part, but keep creating a new configuration	HealthAlarmsPreferenceStore.innerSave	Edit 1 below
Remove the opening of the “Applicable flight” popup	MgsPreferenceDialogHandler.openDialog	Edit 2 below
Create new code to take the deviation values into account	RedAmberAlarmAnalyzer	Part D, done

Edit 1 — the configuration stops carrying edited values

mgs-client/.../preferences/healthalarms/store/HealthAlarmsPreferenceStore.java

The fields stay editable. Editing them is how a deviation is created – the synthesis document is explicit: *“A deviation can be created directly by editing the fields of the indicators list.”* What stops is the write into the new configuration.

WCThresholdBaseline already holds the twelve values as they were loaded, so writing the baseline instead of the model produces a configuration identical to the previous one. The user's change survives only as a deviation.

FIND, in innerSave:

```
for (WCThresholdModel wcThresholdModel : indicatorThresholdModels) {
    if (wcThresholdModel.isEnabled()) {
        final ThresholdConfigurationDto thresholdConfigurationDto;
        if (wcThresholdModel.getType() == ThresholdType.FIXED) {
            thresholdConfigurationDto = new ThresholdConfigurationDto(
                wcThresholdModel.getSeverity(), wcThresholdModel.getLevel(),
                wcThresholdModel.getType(), wcThresholdModel.getDefaultValue(),
                wcThresholdModel.getNbSuccessivePoints(),
                wcThresholdModel.getMinimumEngineOilTemperature());
        } else {
            thresholdConfigurationDto = new ThresholdConfigurationDto(
                wcThresholdModel.getSeverity(), wcThresholdModel.getLevel(),
                wcThresholdModel.getType(), wcThresholdModel.getLearningTime(),
                wcThresholdModel.getMinValue(), wcThresholdModel.getMaxValue(),
                wcThresholdModel.getDefaultValue(), wcThresholdModel.getMinimumDeviation(),
                wcThresholdModel.getMeanFactor(), wcThresholdModel.getDeviationCoefficient(),
                wcThresholdModel.getNbSuccessivePoints(),
                wcThresholdModel.getMinimumEngineOilTemperature());
        }
        thresholdConfigurationDto.setWorkcard(wcThresholdModel.getWorkCardReference());
        indicatorConfigurationDto.getThresholdConfigurationDtos().add(thresholdConfigurationDto);
    }
}
```

REPLACE WITH:

```
for (WCThresholdModel wcThresholdModel : indicatorThresholdModels) {
    if (wcThresholdModel.isEnabled()) {
        // BCR-6605 the indicator values are no longer modified directly. The new
        // configuration carries the values as they were loaded; any change the user
        // made is stored as a deviation instead. The baseline holds those original
        // values, so the configuration comes out unchanged.
        final WCThresholdBaseline baseline = wcThresholdModel.getBaseline();
        final ThresholdType type = baseline != null ? baseline.getType() : wcThresholdModel.getType();
        final ThresholdLevel level = baseline != null ? baseline.getLevel() : wcThresholdModel.getLevel();

        final ThresholdConfigurationDto thresholdConfigurationDto;
        if (type == ThresholdType.FIXED) {
            thresholdConfigurationDto = new ThresholdConfigurationDto(
                wcThresholdModel.getSeverity(), level, type,
                baseline != null ? baseline.getDefaultValue() : wcThresholdModel.getDefaultValue(),
                baseline != null ? baseline.getNbSuccessivePoints() : wcThresholdModel.getNbSuccessivePoints(),
                wcThresholdModel.getMinimumEngineOilTemperature());
        } else {
            thresholdConfigurationDto = new ThresholdConfigurationDto(
                wcThresholdModel.getSeverity(), level, type,
                baseline != null ? baseline.getLearningTime() : wcThresholdModel.getLearningTime(),
                baseline != null ? baseline.getMinValue() : wcThresholdModel.getMinValue(),
                baseline != null ? baseline.getMaxValue() : wcThresholdModel.getMaxValue(),
                baseline != null ? baseline.getDefaultValue() : wcThresholdModel.getDefaultValue(),
                baseline != null ? baseline.getMinimumDeviation() : wcThresholdModel.getMinimumDeviation(),
                baseline != null ? baseline.getMeanFactor() : wcThresholdModel.getMeanFactor(),
                baseline != null ? baseline.getDeviationCoefficient() : wcThresholdModel.getDeviationCoefficient(),
                baseline != null ? baseline.getNbSuccessivePoints() : wcThresholdModel.getNbSuccessivePoints(),
                wcThresholdModel.getMinimumEngineOilTemperature());
        }
        thresholdConfigurationDto.setWorkcard(
            baseline != null ? baseline.getWorkCardReference() : wcThresholdModel.getWorkCardReference());
        indicatorConfigurationDto.getThresholdConfigurationDtos().add(thresholdConfigurationDto);
    }
}
```

No new imports: WCThresholdBaseline, ThresholdType and ThresholdLevel are all already imported in this file. Severity and minimum engine oil temperature keep coming from the model – neither is deviatable, so neither is on the baseline.

Why not simply delete the block. Benoit asks for the configuration to keep being created. If the `ThresholdConfigurationDto` list is left empty, the new configuration may come out with indicator configurations that have no thresholds at all, and then nothing raises an alarm on that aircraft. Writing the baseline cannot do that: the configuration always goes out with exactly the values it already had.

Edit 2 — the Applicable flight popup stops opening

mgs-client/.../presentation/configuration/store/dialog/MgsPreferenceDialogHandler.java

The dialog already pre-selected the most recent flight, so taking that same flight without showing the popup gives exactly what clicking OK without touching the combo gave.

FIND the whole method:

```
public void openDialog(Map<Aircraft, AircraftConfigurationHistoryDto> aircraftConfigurationHistoryMap) {
    boolean okCancel = false;
    MgsPreferenceDialogModel mgsPreferenceDialogModel = new MgsPreferenceDialogModel();
    BundleContext flightBusinessContext = FrameworkUtil.getBundle(IFlightBusiness.class).getBundleContext();
    IFlightBusiness flightBusiness = flightBusinessContext
        .getService(flightBusinessContext.getServiceReference(IFlightBusiness.class));

    for (Entry<Aircraft, AircraftConfigurationHistoryDto> aircraftConfiguration : aircraftConfigurationHistoryMap
        .entrySet()) {
        List<Flight> flights = new ArrayList<>();
        flights.addAll(flightBusiness.getFlights(aircraftConfiguration.getKey().getId(), true));
        flights.sort(new FlightComparator());

        mgsPreferenceDialogModel.setAircraft(aircraftConfiguration.getKey());
        mgsPreferenceDialogModel.setFlights(flights);
        MgsPreferenceDialog mgsPreferenceDialog = new MgsPreferenceDialog(mgsPreferenceDialogModel);
        if (mgsPreferenceDialog.open() == Window.OK) {
            okCancel = true;
            applicableFlight.put(aircraftConfiguration.getValue(),
                mgsPreferenceDialogModel.getSelectedFlight());
        }
        okCancels.put(aircraftConfiguration.getValue(), okCancel);
    }
}
```

REPLACE WITH:

```
public void openDialog(Map<Aircraft, AircraftConfigurationHistoryDto> aircraftConfigurationHistoryMap) {
    BundleContext flightBusinessContext = FrameworkUtil.getBundle(IFlightBusiness.class).getBundleContext();
    IFlightBusiness flightBusiness = flightBusinessContext
        .getService(flightBusinessContext.getServiceReference(IFlightBusiness.class));

    for (Entry<Aircraft, AircraftConfigurationHistoryDto> aircraftConfiguration : aircraftConfigurationHistoryMap
        .entrySet()) {
        List<Flight> flights = new ArrayList<>();
        flights.addAll(flightBusiness.getFlights(aircraftConfiguration.getKey().getId(), true));
        flights.sort(new FlightComparator());

        // BCR-6605 the "Applicable flight" popup is not applicable to deviations and is no longer
        // opened. The configuration takes the flight the dialog used to pre-select - the most
        // recent one - so the result is what clicking OK without touching the combo produced.
        applicableFlight.put(aircraftConfiguration.getValue(), flights.isEmpty() ? null : flights.get(0));
        okCancels.put(aircraftConfiguration.getValue(), true);
    }
}
```

`org.eclipse.jface.window.Window` becomes unused – delete the import. `flights.get(0)` is deliberately the same element the dialog picked at its own line 181, so whichever direction `FlightComparator` sorts, the behaviour is unchanged.

Leave `MgsPreferenceDialog` and `MgsPreferenceDialogComposite` in place. They are dead code now but harmless, and restoring the dialog is one line if that decision is revisited.

A bug this quietly removes. `okCancel` was declared outside the loop and never reset, so with more than one aircraft, once any dialog had been accepted every later one was recorded as accepted even if the user cancelled it. It no longer matters, since the value is always true, but that is what the old code did.

Test

#	Action	Expect
1	Open the Health page, change a threshold value, add a TE, press Apply	No Applicable flight popup appears
2	Confirm the deviation was stored: <code>SELECT * FROM public.indicator_deviation ORDER BY start_date DESC LIMIT 5;</code>	A new row with the changed value
3	Find the new configuration and check the threshold row it created	The old value, not the one just typed
4	Reload the Health page	The row is green and shows the deviated value – it comes from the deviation, not the configuration
5	Delete the deviation from the Deviation List tab, reload	The row returns to the configured value

Step 3 is the one that proves edit 1. Step 5 proves the configuration was never touched – if the old value had been overwritten at Apply, deleting the deviation would leave the edited value behind.